

Documentazione progetto Seduta di laurea

La Mura F., Iosso G.

2026

Contenuti

1. Introduzione
2. Progettazione concettuale
 - 2.1 Dizionario delle entità
 - 2.2 Dizionario delle relazioni
3. Ristrutturazione
 - 3.1 Analisi delle ridondanze
 - 3.2 Eliminazione degli attributi multivalore/composti
 - 3.3 Eliminazione delle generalizzazioni, accorpamenti e partizioni
 - 3.4 Scelta delle chiavi primarie
4. Progettazione logica
5. Progettazione fisica
 - 5.1 Query SQL di creazione tabelle
 - 5.2 Vincoli tramite Trigger

1 Introduzione

Il seguente elaborato descriverà le fasi di una base di dati relazionale di una seduta di laurea, con scopo di gestire i tirocini e le tesi degli studenti e relative richieste di approvazione di esse da parte del docente referente, mostrando il class diagram/ER con le sue istanze e attributi, un dizionario sulle entità e relazioni tra esse.

2 Progettazione concettuale

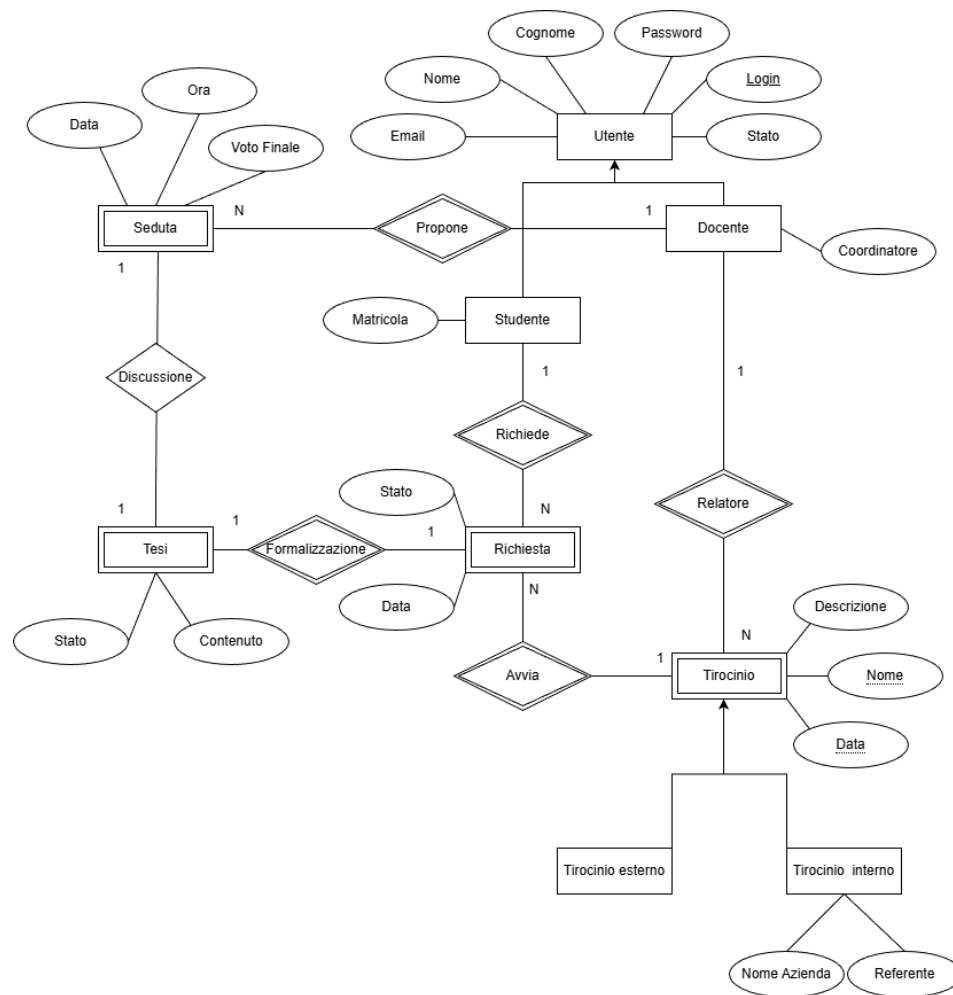


Immagine 1 Entity Relationship(ER)

2.1 Dizionario delle entità

2.1.1 Utente

Traccia il profilo dell'Utente con Nome, Cognome, Password, Login, Email e Loggedin per capire se è un utente che ha loggato.

2.1.2 Studente

Classe figlia di Utente, ha solo Matricola come attributo aggiuntivo.

2.1.3 Docente

Altra classe figlia di Utente, come unico attributo aggiunto è un booleano Coordinatore per segnare se è coordinatore o meno.

2.1.4 Richiesta

Istanza che indica le varie richieste che uno studente può fare sul tirocinio e sulla tesi, i suoi attributi Stato e Data indicano Stato della richiesta (accettato, rifiutato o in attesa)

2.1.5 Tesi

Classe che porta alla tesi dello studente, mostrando lo stato dell'approvazione e il contenuto che sarà un link che porta al documento della tesi, è identificata da Richiesta.

2.1.6 Seduta

Istanza che indica semplicemente la data della seduta di laurea, la tesi discussa e il voto finale, essendo identificata da Docente.

2.1.7 Tirocinio

Classe che rappresenta il tirocinio con descrizione, data e nome del tirocinio, con queste ultime due che sono chiavi parziali.

Ha una relazione identificante con Docente e Richiesta, ciò la rende entità debole.

2.1.8 Tirocinio interno

Classe figlia di Tirocinio, non aggiunge nulla in quanto interno all'università.

2.1.9 Tirocinio esterno

Classe figlia di Tirocinio a cui è aggiunto il nome dell'azienda che offre il tirocinio e il rispettivo referente aziendale.

2.2 Dizionario delle relazioni

2.2.1 Utente-Studente e Utente-Docente

Essendo Studente e Docente specializzazioni di Utente sono legati da una relazione di ereditarietà “is a”, totale e disgiunta. Ogni studente e ogni docente sono utenti, mentre un Utente deve essere o uno studente o un docente.

2.2.2 Studente-Richiesta (Richiede)

La relazione Studente-Richiesta è una delle due relazioni identificanti per Richiesta. Ogni studente può effettuare un numero qualsiasi di richieste mentre che una richiesta deve assolutamente essere correlata a uno e un solo studente.

2.2.3 Richiesta-Tesi (Formalizzazione)

Tesi è un'entità debole e dipende dalla sua relazione con Richiesta. Una richiesta può essere associata ad una tesi, mentre la tesi deve necessariamente essere relativa ad un'unica richiesta.

2.2.4 Tesi-Seduta (Discussione)

Una seduta riguarda la discussione di una tesi, mentre una tesi può essere discussa ma una sola volta.

2.2.5 Richiesta-Tirocinio (Avvia)

Altra relazione identificante per Richiesta. Ogni tirocinio può essere correlato a varie richieste, mentre una richiesta è associata ad un singolo tirocinio obbligatoriamente.

2.2.6 Tirocinio-Docente (Relatore)

Tirocinio è anch'essa debole, motivo per cui la relazione Tirocinio-Docente è identificante. Un docente può organizzare uno o più tirocini, mentre un tirocinio ha un solo professore relatore.

2.2.7 Tirocinio-Tirocinio esterno e Tirocinio-Tirocinio interno

Come per Studente-Docente e Utente-Docente, Tirocinio-Tirocinio esterno e Tirocinio-Tirocinio interno sono relazioni di tipo “is a”, anche in questo caso totali e disgiunte. Un tirocinio può essere solo e solamente interno o esterno. Ogni tirocinio interno/esterno è un tirocinio.

2.2.8 Seduta-Docente (Propone)

Anche la relazione Tesi-Docente è Identificante, in questo caso per Seduta. Una seduta può essere tenuta da un solo docente, mentre un docente può proporre più sedute.

3 Ristrutturazione

3.1 Analisi delle ridondanze

Non ci sono ridondanze apparenti nello schema.

3.2 Eliminazione di attributi multivalore/composti

Non sono presenti attributi multivalore/composti.

3.3 Eliminazione delle generalizzazioni, accorpamenti e partizioni

3.3.1 Utente e Docente/Studente

Utente è un'entità senza relazioni e Docente e Studente svolgono due ruoli completamente diversi all'interno del database. Per questo si ritiene consono di accorpare Utente all'interno delle entità figlie.

3.3.2 Tirocinio e TirocinioInterno/TirocinioEsterno

Visto che Tirocinio come entità partecipa a varie relazioni e solo TirocinioEsterno aggiunge attributi al padre, l'entità TirocinioInterno sarà accorpata nel padre, mentre TirocinioEsterno resterà sullo schema per evitare abbondanze di attributi vuoti in memoria.

3.4 Scelta delle chiavi primarie effettive

3.4.1 Docente e Studente

Visto che Docente e Studente ereditano da Utente ed Utente utilizza come chiave primaria l'attributo Login, sembra consono che anche esse lo utilizzino come chiave primaria.

3.4.2 Tirocinio

Dato che Tirocinio dipende dalla sua relazione con Docente e una chiave "naturale" (Composta dalla relazione, dalla data e dal nome) sarebbe ingombrante per le sue relazioni, si decide di utilizzare una chiave surrogata (nominata ID_Ti) con la quale sarà più semplice gestire chiavi esterne. I problemi relativi alla verifica dell'unicità saranno poi gestiti con dei controlli.

3.4.3 Richiesta

Richiesta è debole con dipendenza da 2 altre entità, Studente e Tirocinio. Per semplificare la gestione del database utilizzeremo anche qui una chiave primaria surrogata (ID_Ri).

3.4.4 Tesi

Un'altra entità debole alla quale verrà assegnata una chiave surrogata (ID_Te).

3.4.5 Seduta

Visto che anche qui una chiave "naturale" sarebbe molto ingombrante, si ricorre anche qui ad una chiave primaria.

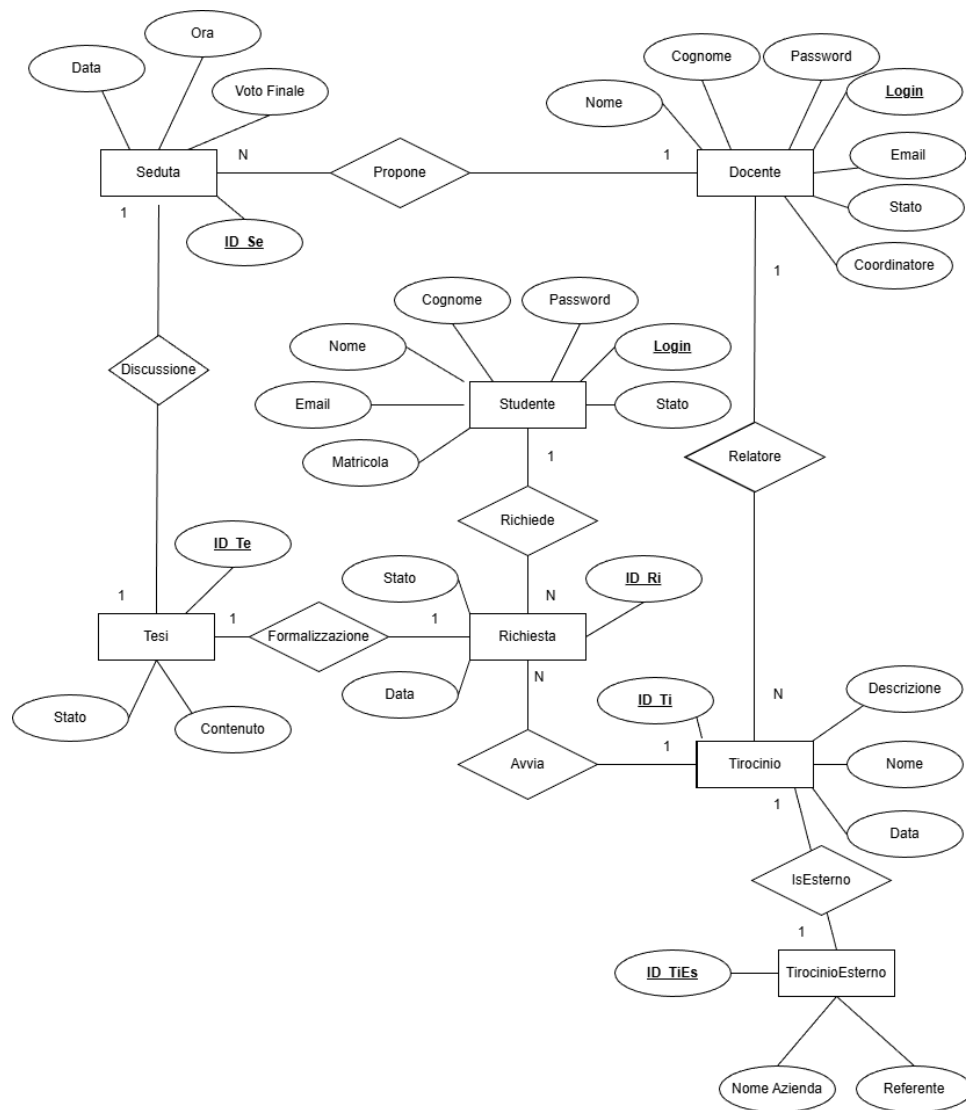


Immagine 2 ER Ristrutturato

4 Progettazione logica

```

STUDENTE(Login, Password, Nome, Cognome, Email, Matricola,
Stato);
DOCENTE(Login, Password, Nome, Cognome, Email, Matricola,
Stato);
TIROCINIO(ID_Ti, Descrizione, Nome, Data, Login^);
TIROCINIO.Login = DOCENTE.Login;
TIROCINIOESTERNO(ID_TiEs, NomeAzienda, Referente, ID_TI^);
TIROCINIOESTERNO.ID_TI = TIROCINIO.ID_TI;
RICHIESTA(ID_Ri, Data, Stato, Login^, ID_Ti^);
RICHIESTA.Login = STUDENTE.Login;
RICHIESTA.ID_Ti = TIROCINIO.ID_Ti;
TESI(ID_Te, Stato, Contenuto, ID_Ri^);
TESI.ID_Ri = RICHIESTA.ID_Ri;
SEDUTA(ID_Se, Data, Ora, VotoFinale, ID_Te^, Login^);
SEDUTA.ID_Te = TESI.ID_Te;
SEDUTA.Login = DOCENTE.Login;

```

5 Progettazione fisica

5.1 Query di creazione tabelle

5.1.1 Studente

```

CREATE TABLE STUDENTE(
    Nome VARCHAR(32) NOT NULL,
    Cognome VARCHAR(32) NOT NULL,
    Password VARCHAR(64) NOT NULL,
    Login VARCHAR(32) PRIMARY KEY,
    Stato BOOLEAN NOT NULL DEFAULT false,
    Email VARCHAR(128) UNIQUE NOT NULL,
    Matricola CHAR(10) UNIQUE NOT NULL,
    CONSTRAINT StudenteEmailCheck CHECK(Email LIKE '%@%.%'),
    CONSTRAINT StudenteLoginLength CHECK(LENGTH(LOGIN)>=4)
);

```

5.1.2 Docente


```

CREATE TABLE DOCENTE(
    Nome VARCHAR(32) NOT NULL,
    Cognome VARCHAR(32) NOT NULL,
    Password VARCHAR(64) NOT NULL,
    Login VARCHAR(32) PRIMARY KEY,
    Stato BOOLEAN NOT NULL DEFAULT false,
    Email VARCHAR(128) UNIQUE NOT NULL,
    Coordinatore BOOLEAN,
    CONSTRAINT DocenteEmailCheck CHECK(Email LIKE '%@%.%'),
    CONSTRAINT DocenteLoginLength CHECK(LENGTH(LOGIN)>=4)
);

```

5.1.3 Tirocinio

```

CREATE TABLE TIROCINIO(
    ID_Ti SERIAL PRIMARY KEY,
    Descrizione VARCHAR(256),
    Nome VARCHAR(64) NOT NULL,
    Data DATE NOT NULL,
    Login VARCHAR(32) NOT NULL,
    FOREIGN KEY (Login) REFERENCES DOCENTE(Login)
);

```

5.1.4 TirocinioEsterno

```

CREATE TABLE TIROCINIOESTERNO(
    ID_TiEs SERIAL PRIMARY KEY,
    ID_Ti INT UNIQUE NOT NULL REFERENCES TIROCINIO(ID_Ti),
    NomeAzienda VARCHAR(64) NOT NULL,
    Referente VARCHAR(64) NOT NULL
);

```

5.1.5 Richiesta

```

CREATE TABLE RICHIESTA(
    ID_Ri SERIAL PRIMARY KEY,
    Data DATE,
    Stato CHAR NOT NULL DEFAULT '?',
    Login VARCHAR(32) NOT NULL REFERENCES STUDENTE(Login),
    ID_Ti INT NOT NULL REFERENCES TIROCINIO(ID_Ti),
    CONSTRAINT CheckStatoRichiesta CHECK(Stato='?' OR Stato='V' OR
Stato='X')
);

```

5.1.6 Tesi

```

CREATE TABLE TESI(
    ID_Te SERIAL PRIMARY KEY,
    Stato CHAR NOT NULL DEFAULT '?',
    Contenuto VARCHAR(2048) UNIQUE,
    ID_Ri INT NOT NULL UNIQUE REFERENCES RICHIESTA(ID_Ri),
    CONSTRAINT CheckStatoTesi CHECK(Stato='?' OR Stato='V' OR
Stato='X')
);

```

5.1.7 Seduta

```

CREATE TABLE SEDUTA(
    ID_Se SERIAL PRIMARY KEY,
    Data DATE NOT NULL,
    Ora TIME NOT NULL,
    VotoFinale INT,
    ID_Te INT NOT NULL UNIQUE REFERENCES TESI(ID_Te),
    Login VARCHAR(32) NOT NULL REFERENCES DOCENTE(Login),
    CONSTRAINT CheckVotoFinaleSeduta CHECK(VotoFinale>0 AND
VotoFinale<=112)
);

```

5.2 Vincoli tramite Trigger

5.2.1 TirocinioUniqueTrigger

```
CREATE OR REPLACE FUNCTION TirocinioUnique()
    RETURNS trigger AS
$TirocinioUnique$
BEGIN
    IF EXISTS(
        SELECT *
        FROM TIROCINIO AS T
        WHERE T.ID_Ti <> NEW.ID_Ti AND T.Nome = NEW.NOME AND
T.Data = NEW.Data AND T.Login = NEW.Login
    )
    THEN RAISE EXCEPTION 'Tirocinio già esistente sotto altro ID';
    END IF;
    RETURN NEW;
END;
$TirocinioUnique$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER TirocinioUniqueTrigger
    BEFORE INSERT OR UPDATE ON Tirocinio
    FOR EACH ROW
EXECUTE FUNCTION TirocinioUnique();
```

5.2.2 RichiestaUniqueTrigger

```
CREATE OR REPLACE FUNCTION RichiestaUnique()
    RETURNS trigger AS
$RichiestaUnique$
BEGIN
    IF EXISTS(
        SELECT *
        FROM Richiesta AS R
        WHERE R.ID_Ri <> NEW.ID_Ri AND R.ID_Ti = NEW.ID_Ti
        AND R.Data = NEW.Data AND R.Login = NEW.Login
    )
        THEN RAISE EXCEPTION 'Richiesta già esistente sotto altro ID';
    END IF;
    RETURN NEW;
END;
$RichiestaUnique$ LANGUAGE plpgsql;
CREATE OR REPLACE TRIGGER RichiestaUniqueTrigger
    BEFORE INSERT OR UPDATE ON Richiesta
    FOR EACH ROW
EXECUTE FUNCTION RichiestaUnique();
```

5.2.3 RichiestaInCorsoTrigger

```
CREATE OR REPLACE FUNCTION RichiestaInCorso()
    RETURNS trigger AS
$RichiestaInCorso$
BEGIN
    IF EXISTS(
        SELECT *
        FROM Richiesta AS R
        WHERE R.ID_Ri <> NEW.ID_Ri AND R.ID_Ti = NEW.ID_Ti AND
        R.Login = NEW.Login AND ((NEW.Stato != 'X' AND NEW.Data<R.Data)
        OR(R.Stato!='X' AND R.Data<NEW.Data))
    )
        THEN RAISE EXCEPTION 'Richiesta già esistente sotto altro ID';
    END IF;
    RETURN NEW;
END;
$RichiestaInCorso$ LANGUAGE plpgsql;
CREATE OR REPLACE TRIGGER RichiestaInCorsoTrigger
    BEFORE INSERT OR UPDATE ON Richiesta
    FOR EACH ROW
EXECUTE FUNCTION RichiestaInCorso();
```

5.2.4 TesiSuRichiestaAccettata

```
CREATE OR REPLACE FUNCTION TesiSuRichiestaAccettata()  
    RETURNS trigger AS  
$TesiSuRichiestaAccettata$  
BEGIN  
    IF EXISTS(  
        SELECT *  
        FROM Richiesta as R  
        WHERE R.ID_Ri = NEW.ID_Ri AND R.Stato <> 'V'  
    )  
        THEN RAISE EXCEPTION 'Tesi associata a richiesta non  
accettata';  
    END IF;  
  
    RETURN NEW;  
END;  
$TesiSuRichiestaAccettata$ LANGUAGE plpgsql;  
CREATE OR REPLACE TRIGGER TesiSuRichiestaAccettataTrigger  
    BEFORE INSERT OR UPDATE ON Tesi  
    FOR EACH ROW  
EXECUTE FUNCTION TesiSuRichiestaAccettata();
```

5.2.5 SedutaSuTesiAccettata

```
CREATE OR REPLACE FUNCTION SedutaSuTesiAccettata()  
    RETURNS trigger AS  
$SedutaSuTesiAccettata$  
BEGIN  
    IF EXISTS(  
        SELECT *  
        FROM Tesi AS T  
        WHERE T.ID_Te = NEW.ID_Te AND T.Stato <> 'V'  
    )  
    THEN RAISE EXCEPTION 'Seduta associata a tesi non accettata';  
    END IF;  
  
    RETURN NEW;  
END;  
$SedutaSuTesiAccettata$ LANGUAGE plpgsql;  
  
CREATE OR REPLACE TRIGGER SedutaSuTesiAccettataTrigger  
    BEFORE INSERT OR UPDATE ON SEDUTA  
    FOR EACH ROW  
EXECUTE FUNCTION SedutaSuTesiAccettata();
```

5.2.6 SeduteSovrapposizioneDocente

```
CREATE OR REPLACE FUNCTION SeduteSovrapposizioneDocente()
    RETURNS trigger AS
$SeduteSovrapposizioneDocente$
BEGIN
    IF EXISTS(
        SELECT *
        FROM SEDUTA AS S
        WHERE S.Login = NEW.Login
            AND S.Data = NEW.Data
            AND S.Ora = NEW.Ora
            AND S.ID_Se <> COALESCE(NEW.ID_Se, -1)
    )
    THEN RAISE EXCEPTION 'Docente già occupato per seduta';
    END IF;

    RETURN NEW;
END;
$SeduteSovrapposizioneDocente$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER SeduteSovrapposizioneDocenteTrigger
    BEFORE INSERT OR UPDATE ON SEDUTA
    FOR EACH ROW
EXECUTE FUNCTION SeduteSovrapposizioneDocente();
```


5.2.7 RichiestaAttiva

```
CREATE OR REPLACE FUNCTION RichiestaAttiva()
    RETURNS trigger AS
$RichiestaAttiva$
BEGIN
    -- Verifica se lo studente ha già una richiesta in stato '?' o 'V'
    IF EXISTS(
        SELECT *
        FROM RICHIESTA AS R
        WHERE R.Login = NEW.Login
        AND R.Stato <> 'X'
        AND R.ID_Ri <> COALESCE(NEW.ID_Ri, -1)
    )
    THEN RAISE EXCEPTION 'Altra richiesta ancora attiva/in
valutazione>/accettata';
    END IF;

    RETURN NEW;
END;
$RichiestaAttiva$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER RichiestaAttivaTrigger
    BEFORE INSERT OR UPDATE ON RICHIESTA
    FOR EACH ROW
EXECUTE FUNCTION RichiestaAttiva();
```

5.2.8 SedutaControlloDataRichiesta

```
CREATE OR REPLACE FUNCTION SedutaControlloDataRichiesta()  
    RETURNS trigger AS  
$SedutaControlloDataRichiesta$  
DECLARE  
    data_richiesta DATE;  
BEGIN  
    SELECT R.Data INTO data_richiesta  
    FROM TESI Te  
    JOIN RICHIESTA R ON Te.ID_Ri = R.ID_Ri  
    WHERE Te.ID_Te = NEW.ID_Te;  
  
    IF NEW.Data <= data_richiesta THEN  
        RAISE EXCEPTION 'Seduta precedente rispetto alla richiesta di  
tirocinio.';  
    END IF;  
  
    RETURN NEW;  
END;  
$SedutaControlloDataRichiesta$ LANGUAGE plpgsql;  
  
CREATE OR REPLACE TRIGGER SedutaControlloDataRichiestaTrigger  
    BEFORE INSERT OR UPDATE ON SEDUTA  
    FOR EACH ROW  
EXECUTE FUNCTION SedutaControlloDataRichiesta();
```

5.2.9 TesiRimuoviAnnullamentoRichiesta

```
CREATE OR REPLACE FUNCTION TesiRimuoviAnnullamentoRichiesta()  
    RETURNS trigger AS  
$TesiRimuoviAnnullamentoRichiesta$  
BEGIN  
    IF NEW.Stato = 'X' AND OLD.Stato = 'V' THEN  
        DELETE FROM SEDUTA  
        WHERE ID_Te IN (SELECT ID_Te FROM TESI WHERE ID_Ri =  
NEW.ID_Ri);  
        DELETE FROM TESI  
        WHERE ID_Ri = NEW.ID_Ri;  
    END IF;  
  
    RETURN NEW;  
END;  
$TesiRimuoviAnnullamentoRichiesta$ LANGUAGE plpgsql;  
  
CREATE OR REPLACE TRIGGER TesiRimuoviAnnullamentoRichiestaTrigger  
    AFTER UPDATE ON RICHIESTA  
    FOR EACH ROW  
EXECUTE FUNCTION TesiRimuoviAnnullamentoRichiesta();
```

5.2.10 SedutaRimuoviAnnullamentoTesi

```
CREATE OR REPLACE FUNCTION SedutaRimuoviAnnullamentoTesi()  
    RETURNS trigger AS  
$SedutaRimuoviAnnullamentoTesi$  
BEGIN  
    IF NEW.Stato = 'X' AND OLD.Stato = 'V' THEN  
        DELETE FROM SEDUTA  
        WHERE ID_Te = NEW.ID_Te;  
    END IF;  
  
    RETURN NEW;  
END;  
$SedutaRimuoviAnnullamentoTesi$ LANGUAGE plpgsql;  
  
CREATE OR REPLACE TRIGGER SedutaRimuoviAnnullamentoTesiTrigger  
    AFTER UPDATE ON TESI  
    FOR EACH ROW  
EXECUTE FUNCTION SedutaRimuoviAnnullamentoTesi();
```

5.2.11 SedutaAggiungiVotoSuTesi

```
CREATE OR REPLACE FUNCTION SedutaAggiungiVotoSuTesi()  
    RETURNS trigger AS  
$SedutaAggiungiVotoSuTesi$  
BEGIN  
    IF ISNOTNULL(NEW.VotoFinale) AND ISNULL(NEW.ID_Te)  
        THEN RAISE EXCEPTION 'Tesi mancante per voto';  
    END IF;  
    RETURN NEW;  
END  
$SedutaAggiungiVotoSuTesi$ LANGUAGE plpgsql;  
  
CREATE OR REPLACE TRIGGER SedutaAggiungiVotoSuTesiTrigger  
    BEFORE UPDATE ON Tesi  
    FOR EACH ROW  
EXECUTE FUNCTION SedutaAggiungiVotoSuTesi();
```

5.2.12 SedutaControlloDataTirocinio

```
CREATE OR REPLACE FUNCTION SedutaControlloDataTirocinio()
    RETURNS trigger AS
$SedutaControlloDataTirocinio$
DECLARE
    data_tirocinio DATE;
BEGIN
    SELECT T.Data INTO data_tirocinio
    FROM TESI Te
    JOIN RICHIESTA R ON Te.ID_Ri = R.ID_Ri
    JOIN TIROCINIO T ON R.ID_Ti=T.ID_Ti
    WHERE Te.ID_Te = NEW.ID_Te ;

    IF NEW.Data <= data_tirocinio THEN
        RAISE EXCEPTION 'Seduta precedente rispetto alla richiesta di
tirocinio.';
    END IF;

    RETURN NEW;
END;
$SedutaControlloDataTirocinio$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER SedutaControlloDataTirocinioTrigger
    BEFORE INSERT OR UPDATE ON Seduta
    FOR EACH ROW
EXECUTE FUNCTION SedutaControlloDataTirocinio();
```